Design Document

2024-10189 한재준

1. New Data Structures

1-1. kalloc.c: struct frame_group

이번 과제4에서는 4 KiB frame과 2 MiB huge frame을 같은 물리 메모리에서 할당하기 위해, 기존 freelist 대신 frame_group 배열을 사용하였다. 물리 메모리 128 MiB를 2 MiB 단위의 64개 group으로 나누고, 각 group을 다시 512개의 4 KiB frame으로 관리한다.

① used[512], used_count: 각 4 KiB frame의 사용 여부와 group에서 사용 중인 frame 수를 저장함.

② huge_allocated: group 전체가 하나의 huge frame으로 할당되었는지를 저장함. 같은 주소라도 일반 frame 512개와 huge frame 1개는 다르게 해제해야 하기 때문에 필요함.

③ ref[512], huge_ref: 일반 frame별 참조 수와 huge frame의 참조 수를 각각 저장함. fork() 후 여러 process가 같은 물리 frame을 사용하므로 마지막 참조가 사라질 때만 실제로 반환함.

주소 pa에 대해 offset = pa - KERNBASE, group = offset / HGPGSIZE, index = (offset % HGPGSIZE) / PGSIZE로 위치를 구한다. kmem.lock으로 사용 상태, 참조 수, 메모리 카운터를 함께 보호하여 여러 CPU가 같은 frame을 동시에 할당하지 않도록 하였다.

1-2. proc.h: struct mmap_region, struct mmap_object

① mmap_region: valid, addr, length, prot, flags, obj를 저장함. proc의 regions[4]에서 process별 mmap 영역을 관리하며, length에는 page 크기에 맞게 올림한 길이를 저장함.

② mmap_object: refs, lock, pages, length를 저장함. fork()로 상속한 영역이 같은 object를 가리키도록 하여 process의 영역 정보와 공유 상태를 구분함.

object의 pages는 영역 시작점으로부터의 offset을 물리 frame에 대응시키는 내부 page table이다. MAP_SHARED에서만 실제 frame 표로 사용하며 첫 fault 때 생성한다. 부모가 한 번도 접근하지 않은 상태로 fork()하더라도, 자식이 먼저 만든 frame을 부모가 나중에 찾을 수 있도록 하기 위해 필요하다. 이 표는 사용자 page table과 별개이며 PTE_U를 설정하지 않는다.

1-3. 전역 상태와 PTE_COW

sysproc.c의 mmap_state에는 lock, count, objects[64]를 두었다. object의 refs는 해당 object를 가리키는 영역의 수이며, fork()는 이를 증가시키지만 서로 다른 mmap 영역의 전역 개수를 늘리지는 않는다. 마지막 영역이 해제되면 object를 재사용할 수 있다. 또한 PTE의 software 전용 bit 8을 PTE_COW로 정의하여, 원래부터 읽기 전용인 page와 COW 때문에 쓰기가 금지된 page를 구분하였다.

2. Algorithm Design

2-1. kalloc_huge(), kfree_huge()

kalloc_huge()는 used_count가 0인 group을 찾아 전체를 할당한다. KERNBASE와 group 크기가 모두 2 MiB에 정렬되어 있으므로 반환 주소도 2 MiB에 정렬된다. kfree_huge()는 주소와 할당 종류를 확인하고 참조 수가 0이 되었을 때 group 전체를 반환한다. 아래 의사코드의 P는 512이다.

```
kalloc_huge():
  lock(kmem)
  g = first group with used_count == 0
  if no such group: unlock(kmem); return 0
  g.used[:] = 1; g.used_count = P
  g.huge_allocated = 1; g.huge_ref = 1
  freemem -= P; used2m += 1
  unlock(kmem)
  pa = KERNBASE + group_number(g) * HGPGSIZE
  fill(pa, 5, HGPGSIZE)
  return pa

kfree_huge(pa):
  if pa is unaligned or outside the freeable range: panic
  lock(kmem); g = group_of(pa)
  if not g.huge_allocated or g.huge_ref <= 0: panic
  g.huge_ref -= 1
  if g.huge_ref > 0: unlock(kmem); return
  g.used[:] = 0; g.used_count = 0; g.huge_allocated = 0
  freemem += P; used2m -= 1
  fill(pa, 1, HGPGSIZE)
  unlock(kmem)
```

기존 kalloc()은 먼저 일부만 사용 중인 group에서 빈 4 KiB frame을 찾고, 그런 group이 없을 때만 완전히 빈 group을 사용하도록 바꾸었다. 작은 할당을 모아서 빈 group을 최대한 남기기 위해서이다. kfree()도 참조 수가 0일 때만 used와 used_count를 바꾸므로, 같은 group의 frame 512개가 모두 반환되면 별도의 병합 없이 huge frame으로 다시 사용할 수 있다.

kinit()에서는 우선 모든 frame을 사용 중으로 표시하고, freerange(end, PHYSTOP)에서 kernel 이후의 frame만 반환한다. 따라서 kernel이 차지한 공간을 잘못 할당하지 않으며, 초기 카운터에서도 kernel 공간은 used4k에 포함된다. 할당기의 fill 값은 디버깅용이고, mmap의 zero 초기화는 fault 처리 시 별도로 수행한다.

2-2. mmap(), munmap()

sys_mmap()은 length 범위, prot, flags, 주소 정렬을 검사하고 길이를 page 크기로 올림한다. 이후 기존 영역과 [addr, addr + rounded)의 겹침 여부, process별 4개 제한, 전역 object 64개 제한을 확인한다. 모든 검사가 끝나면 영역 정보만 기록하고 addr를 반환한다. 이 시점에는 데이터 frame이나 page table을 할당하지 않는다. 실패 시에는 0을 반환하며 영역을 남기지 않는다.

sys_munmap()은 addr가 valid 영역의 시작 주소와 정확히 같은 경우에만 영역 전체를 해제한다. 사용자 PTE의 참조를 먼저 반환한 뒤 valid를 지우고 object의 참조도 반환한다. 해당 시작 주소가 없으면 -1을 반환한다. 아직 접근하지 않은 page는 PTE가 없으므로 건너뛴다.

2-3. Page Table과 Page Fault Handler

기존 walk()는 L0까지 내려가지만 huge page는 L1이 leaf이다. 따라서 walklevel()을 추가하여 R/W/X bit가 있는 leaf를 만나면 멈추고 level도 반환하도록 하였다. 일반 page는 L0, huge page는 L1 PTE를 사용하며, maphugepages()는 L1에 직접 leaf를 만든다. walkaddr()는 huge frame 내부의 4 KiB 단위 offset까지 더해 copyin(), copyout(), copyinstr()가 기존 복사 루프를 사용할 수 있도록 하였다.

vmfault()에서는 mmap 주소가 p->sz보다 클 수 있으므로 먼저 regions를 확인한다. 전달받은 page table이 현재 process의 page table일 때만 해당 영역 정보를 사용하여, exec()가 새 주소 공간을 만드는 중에 이전 mmap을 잘못 참조하지 않도록 하였다.

```
vmfault(pt, psz, va, read):
  r = valid mmap region containing va, if pt == current.pt
  if r exists:
    if store and r.prot == PROT_READ: return 0
    size = r has MAP_HUGEPAGE ? 2MiB : 4KiB
    va0 = align_down(va, size)
    if a valid user mapping already exists:
      if store and r is MAP_PRIVATE:
        if PTE_COW is absent: return 0
        if old frame has one reference: enable W; clear COW
        else: allocate same-size frame; if none: return 0
              copy all bytes; replace PTE with W and without COW
        flush TLB
        if a new frame was used: drop old frame reference
        return resolved physical page
      return 0
    if r is MAP_SHARED: return shared_fault(r, va0)
    allocate a frame of size; if allocation fails: return 0
    zero the whole frame; map with U | R and optional W
    if mapping fails: free the frame; return 0
    return resolved physical page
  if va >= psz or already mapped: return 0
  allocate, zero, and map the original 4KiB lazy heap page
  on failure: release any new frame; return 0
  return resolved physical page
```

읽기 fault도 새 frame을 할당하여 전체를 0으로 채운다. 하나의 zero page를 공통으로 매핑하지 않으며, huge 영역에서는 접근한 주소를 포함하는 2 MiB 전체를 한 번에 할당한다. PROT_WRITE에는 R과 W를 모두 설정하고, PROT_READ에는 R만 설정한다. 두 경우 모두 U를 설정하고 실행 권한 X는 주지 않는다.

```
shared_fault(r, va0):
  lock(r.obj); offset = va0 - r.addr
  look up offset; create the object's table lazily if needed
  if absent: allocate, zero, and register one same-size frame
  if table/frame allocation or registration fails:
    free any unregistered new frame; unlock(r.obj); return 0
  add one frame reference for the process PTE
  unlock(r.obj)
  map that frame in the process page table
  if mapping fails: drop the added reference; return 0
  return resolved physical page
```

공유 표의 조회와 최초 할당을 같은 object lock 안에서 수행하여 동시에 fault가 발생해도 frame이 중복 생성되지 않도록 하였다. 공유 표 자체도 frame 참조를 하나 보유하므로, 자식이 먼저 쓰고 종료한 뒤 부모가 처음 접근하는 경우에도 데이터가 유지된다. 메모리 부족이나 권한 위반은 0을 반환하여 usertrap()이 해당 process를 종료하게 하며, 정상적인 자원 부족으로 kernel panic이 발생하지 않게 하였다.

2-4. fork()와 COW

kfork()에서는 기존 uvmcopy()로 code, data, stack 및 할당된 heap page를 복사한 다음 mmapfork()를 호출한다. lazy heap의 미할당 page는 그대로 두며 heap에는 huge page를 사용하지 않는다. mmap 영역에는 아래 절차를 적용한다. child의 매핑이 모두 성공한 뒤 parent의 PTE를 COW로 바꾸므로, 중간 실패 시 parent의 권한을 복구하는 작업이 필요하지 않다.

```
mmapfork(parent, child):
  for each valid region r in parent:
    copy r to child; hold(r.obj)
    for each page-sized position in r:
      if parent has no physical mapping: continue
      pa, perm = parent's frame and PTE flags
      if r is MAP_PRIVATE and PROT_WRITE:
        perm = (perm without W) with COW
      add frame reference
      map the same frame in child using r's page size
      if mapping fails:
        drop the just-added frame reference
        clean up all child's mmap regions; return -1
  for each mapped page in parent's writable private regions:
    clear W; set COW
  flush TLB; return 0
```

① MAP_SHARED: 이미 할당된 frame은 동일한 권한으로 공유한다. 아직 할당되지 않은 page는 부모와 자식 모두 PTE 없이 두되, 같은 object의 pages를 통해 이후 최초 fault에서도 같은 frame을 사용한다.

② MAP_PRIVATE + PROT_WRITE: 이미 할당된 frame만 우선 공유하고 양쪽에 COW를 설정한다. 이후 쓰기 fault가 난 process만 새 frame으로 분리된다. 처음부터 미할당인 page는 각 process가 나중에 독립적으로 할당한다.

③ MAP_PRIVATE + PROT_READ: 같은 frame을 읽기 전용으로 공유하되 COW는 설정하지 않는다. 따라서 store가 발생해도 쓰기 가능한 복사본을 만들지 않고 해당 process를 종료한다.

COW fault에서 참조 수가 1이면 다른 사용자가 없으므로 복사 없이 W를 복원하고 COW를 지운다. 참조가 여러 개이면 동일한 크기의 새 frame에 전체 내용을 복사하고 PTE를 교체한 뒤 이전 참조를 반환한다. huge page의 경우 항상 2 MiB 전체를 복사하며 L1 leaf를 유지한다. PTE가 바뀐 뒤에는 sfence_vma()로 이전 주소 변환을 무효화한다.

또한 read() 등으로 kernel이 사용자 buffer에 쓰는 경우에는 사용자 store fault가 발생하지 않는다. 따라서 copyout()에서도 PTE_COW를 확인하여 같은 COW 처리 함수를 호출하고, 갱신된 물리 주소를 다시 구한 뒤 복사하도록 하였다. 이 처리가 없으면 자식의 read()가 부모의 private frame을 직접 바꾸거나 정상적인 COW buffer 쓰기가 실패할 수 있다.

2-5. 해제와 참조 수 관리

uvmunmap()은 leaf level에 따라 kfree() 또는 kfree_huge()를 호출한다. 미할당 page는 건너뛰고, huge leaf를 만난 경우 다음 2 MiB 경계로 이동한다. frame 참조를 반환하는 것과 실제 물리 메모리를 반환하는 것을 구분하여, 다른 process가 사용하는 frame을 해제하지 않도록 하였다.

MAP_SHARED에서는 각 사용자 PTE와 object의 공유 표가 각각 참조를 가진다. 마지막 영역의 object 참조가 사라지면 공유 표의 데이터 frame과 page-table page를 함께 반환한다. mmap_cleanup()을 exit() 및 성공한 exec()의 주소 공간 교체 직전에 호출하여 영역 전체를 정리한다. 실패한 exec()는 기존 주소 공간과 mmap을 유지한다. object의 최종 정리가 끝날 때까지 전역 lock을 유지하여 다른 CPU가 정리 중인 object를 재사용하지 못하게 하였다.

2-6. Corner Cases

① 잘못된 입력: length가 0 이하 또는 64 MiB 초과인 경우, 정렬 위반, 잘못된 prot, PRIVATE/SHARED의 동시 지정 또는 누락, 정의되지 않은 flag를 거부함. 겹침은 올림한 길이를 기준으로 검사하며, munmap()은 시작 주소만 허용함.

② 최초 접근 전 fork(): SHARED는 공통 object에 frame을 등록하여 늦게 접근한 process도 같은 내용을 보게 하고, PRIVATE는 각자 처음 접근할 때 별도 frame을 할당함.

③ 권한 위반과 자원 부족: PROT_READ에 대한 store, frame 또는 page table 할당 실패는 fault 처리 실패로 반환함. fork()의 중간 실패는 child의 PTE와 object 참조를 정리함.

④ 마지막 참조와 미할당 영역 해제: 물리 frame 참조가 0일 때만 카운터와 사용 상태를 바꿈. 접근한 적 없는 영역은 데이터 frame 없이 영역 정보만 정리함.

⑤ page 크기를 바꾸어 같은 주소 재사용: 일반 page를 unmap한 뒤 빈 L0 table이 남아 있으면, maphugepages()가 그 table의 모든 PTE가 비어 있는지 확인한 뒤 L1 huge leaf로 교체하고 L0 table을 반환함. 유효한 매핑이 남아 있으면 교체하지 않음.

⑥ 다중 CPU: allocator 정보는 kmem.lock, object 수명은 mmap_state.lock, 공유 frame 조회와 생성은 object lock으로 보호함. pagefaults는 atomic 연산으로 증가시키며, 유효한 mmap 영역에서 발생한 실제 load/store trap만 계수함.

2-7. Optimization

4 KiB 할당을 일부 사용 중인 group에 우선 배치하여 2 MiB 단위의 빈 공간을 보존하였다. G를 group 수, P를 group당 일반 frame 수라 하면 일반 할당의 탐색 비용은 O(G + P), huge 할당의 탐색 비용은 O(G)이다. huge 할당과 최종 해제는 P개 사용 표시를 갱신하고 2 MiB를 채우는 비용이 추가된다. 관리 배열의 공간 비용은 O(GP)이며, 현재 G = 64, P = 512로 고정되어 있다.

mmap은 영역 정보만 저장하고 공유 표도 첫 fault 때 생성하므로 사용하지 않는 데이터 frame을 할당하지 않는다. 공유 표는 기존 page-table 구조를 사용하여 offset 조회를 고정된 깊이 안에서 처리한다. huge page는 L0 table 없이 L1 leaf로 표현하여 page-table 공간을 줄인다. PRIVATE fork()도 데이터 복사를 뒤로 미루며, COW의 마지막 소유자는 새 frame 할당을 생략한다. 다만 mmapfork()는 미할당 page 위치도 순회하므로 영역의 page 수에 비례하는 탐색 비용은 남는다.

3. Testing and Validation

3-1. 제공된 mmaptest

4 CPU, 128 MiB QEMU 환경에서 mmaptest1, mmaptest2, mmaptest3을 실행하였다. mmaptest1은 huge shared 영역에 기록한 DEADBEEF를 읽었고, mmaptest2는 mmap 직후 used2m = 0, 최초 store 후 used2m = 1이 되었다. 자식이 기록한 BADBEEF가 부모에서도 보이는 것을 확인하였다. mmaptest3에서는 손자만 BADBEEF로 바뀌고 부모와 자식은 DEADBEEF를 유지하여 private COW가 적용되는 것을 확인하였다.

3-2. Normal Process Test와 메모리 카운터

forktest에서 fork test OK, usertests -q에서 ALL TESTS PASSED를 확인하였다. 또한 cat README | wc, cat README | wc | wc | wc, cat | cat | cat을 실행하여 일반 process 생성, pipe 입출력 및 종료가 정상적으로 동작하는지 확인하였다.

shell의 메모리 사용이 안정된 상태에서 측정한 (freemem, used4k, used2m)은 (32496, 272, 0)이었다. 제공된 mmaptest, 추가 검증, 일반 회귀 테스트가 종료되어 shell로 돌아왔을 때 같은 값으로 복구되었다. 이때 32496 + 272 + 512 × 0 = 32768로 카운터 관계도 만족하였다. pagefaults는 누적값이므로 복구 대상에서 제외하였다.

3-3. Corner Case Validation

기본 예제만으로 확인하기 어려운 동작은 다음 6개 주제로 추려 별도로 검증하였다. 원본 소스에 test 코드를 추가하지 않고 임시 복사본에 검증 프로그램을 빌드하였다. page 크기나 공유 방식에 따른 차이가 있는 경우 4 KiB와 2 MiB, PRIVATE와 SHARED 조합을 함께 확인하였고, 아래 항목은 모두 통과하였다.

① 인자와 영역 검사: 잘못된 length, prot, flags, 정렬 및 영역 중첩을 거부하는지 확인함. 올림한 길이까지 포함하여 겹침을 판단하고, process별 4개 제한 및 잘못된 munmap() 주소도 검사함.

② 읽기 최초 접근과 지연 할당: 세 page 크기의 영역에서 처음 읽은 값이 0인지 확인함. 같은 page의 재접근은 fault를 추가하지 않고, 다른 page의 최초 접근에서만 fault가 증가함. huge 영역에서는 접근한 두 page에 대해서만 used2m이 2 증가하고 나머지 page는 미할당으로 남음.

③ 미접근 SHARED 영역을 fork(): 부모가 접근하기 전에 자식이 두 번째 page에 값을 쓰고 종료하게 함. 부모가 그 주소를 처음 읽어도 자식의 값이 남아 있어, 공유 표의 frame 보존과 늦은 매핑이 동작함을 확인함.

④ PRIVATE COW와 단독 소유: frame의 처음, 중간, 마지막에 서로 다른 값을 넣은 뒤 자식이 일부를 수정하게 함. 자식의 나머지 내용은 유지되고 부모의 내용은 바뀌지 않음을 확인함. 자식 종료 후 부모가 쓰는 경우에는 used4k와 used2m이 증가하지 않아 불필요한 복사를 생략함을 확인함.

⑤ copyout()의 COW 처리: fork()한 자식에서 pipe의 read() 목적지로 private 영역을 전달함. 자식의 buffer만 갱신되고 부모의 값은 유지됨. huge 영역은 3 × PGSIZE + 13 위치를 사용하여 첫 4 KiB 밖의 물리 주소 계산도 확인함.

⑥ 같은 주소에서 page 크기 변경: 일반 page 사용 후 munmap()하여 같은 주소를 huge page로 다시 매핑하고, 반대 순서도 검사함. PRIVATE와 SHARED 모두 재접근이 성공하여 빈 L0 table 때문에 huge 매핑이 막히지 않음을 확인함.

3-4. Most Time-Consuming Part

이번 과제에서 가장 시간이 오래 걸린 부분은 fork() 부분을 작성하는 것이었다. 기존의 uvmcopy()처럼 물리 메모리를 복사하는 것과 달리, mmap 영역은 MAP_SHARED와 MAP_PRIVATE에 따라 서로 다르게 처리해야 했기 때문이다. 특히 부모가 아직 접근하지 않은 영역을 상속하는 경우와 이미 할당된 PRIVATE frame을 COW로 전환하는 경우를 함께 고려해야 했다. 부모와 자식의 PTE를 바꾸는 시점, frame 및 object의 참조 수, 중간 실패 시 반환할 자원을 구분하는 과정이 복잡했다. 따라서 fork()가 성공하는 경로뿐 아니라 자식이 먼저 종료하거나 매핑 도중 실패하는 경로도 나누어 정리하였다.